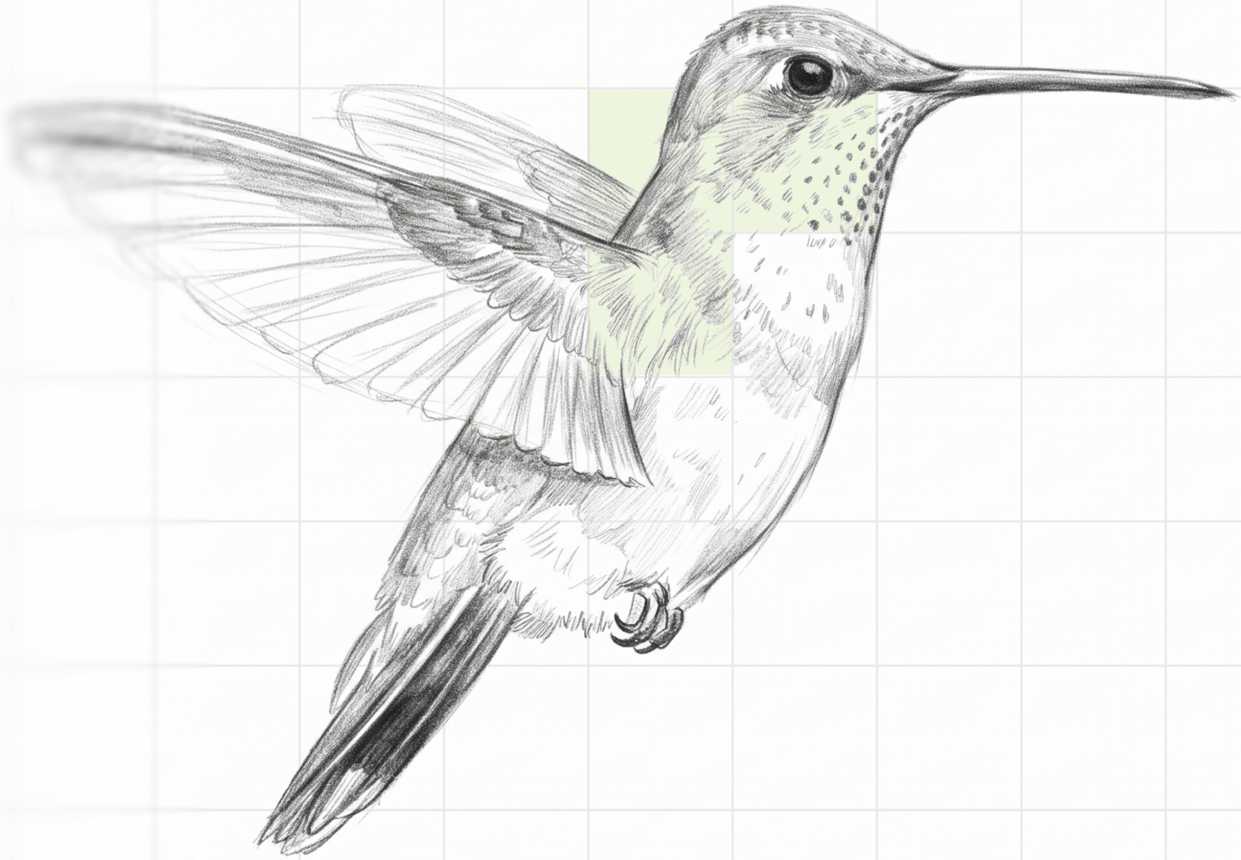
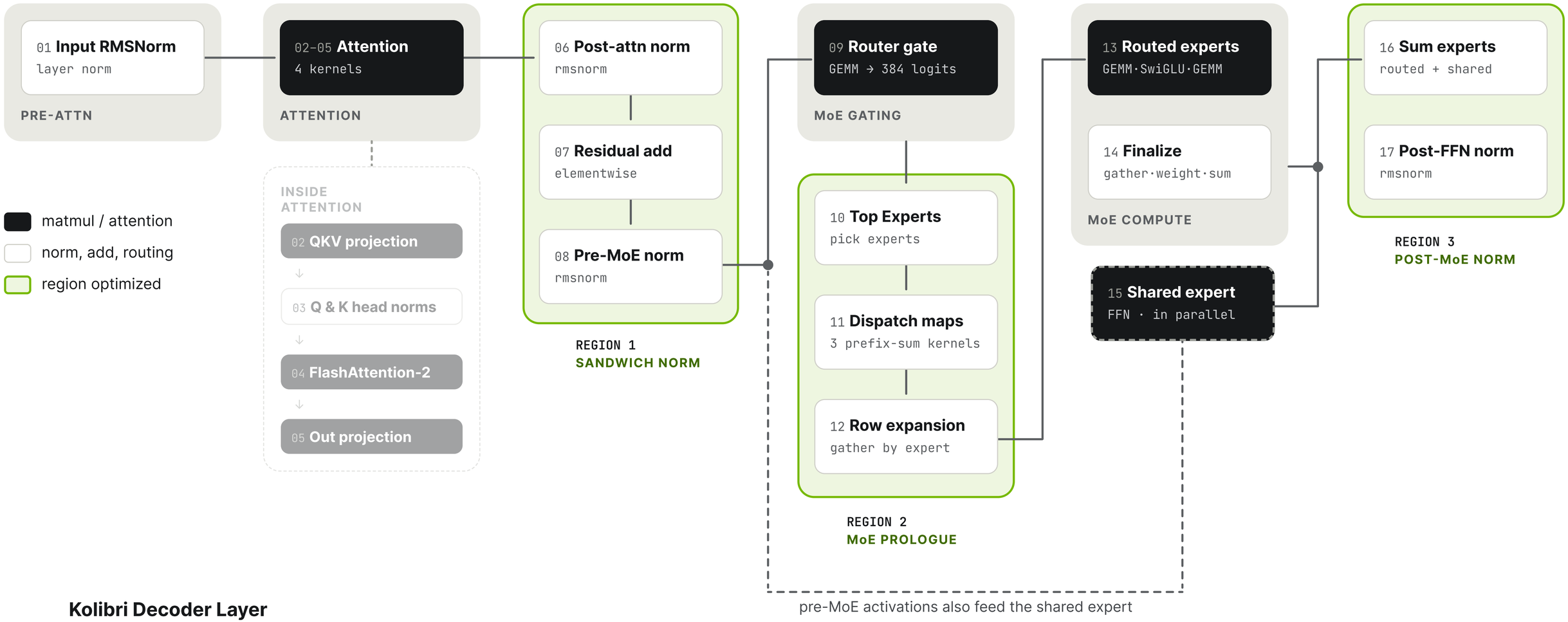
TECHNICAL PRESENTATION

Kolibri Inference Optimization

By Amine Kina
M.Sc. at University of Bremen



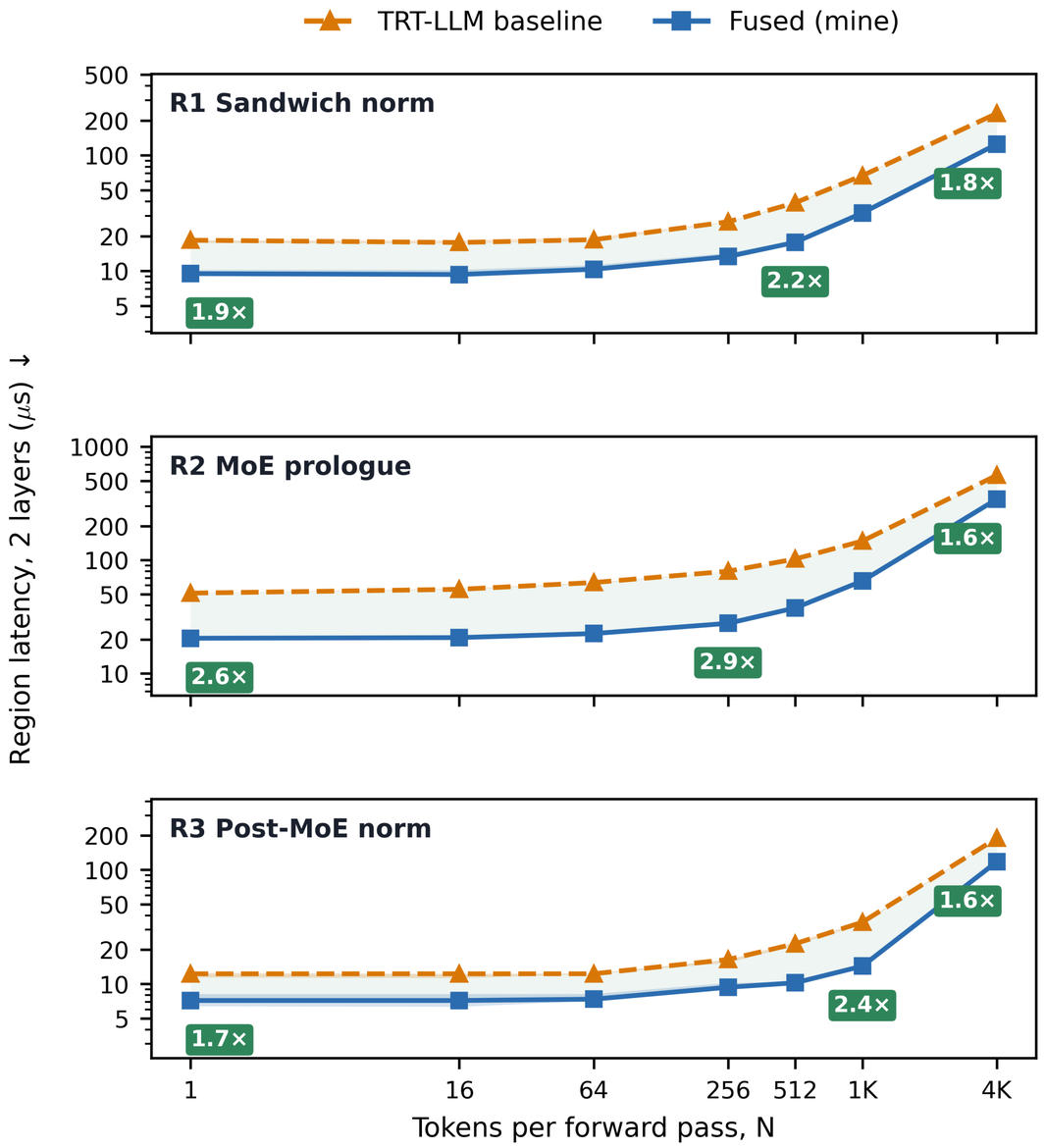
01 Kolibri Architecture



Kolibri Decoder Layer

Repeated across 50 layers → every microsecond saved here scales 50× end-to-end.

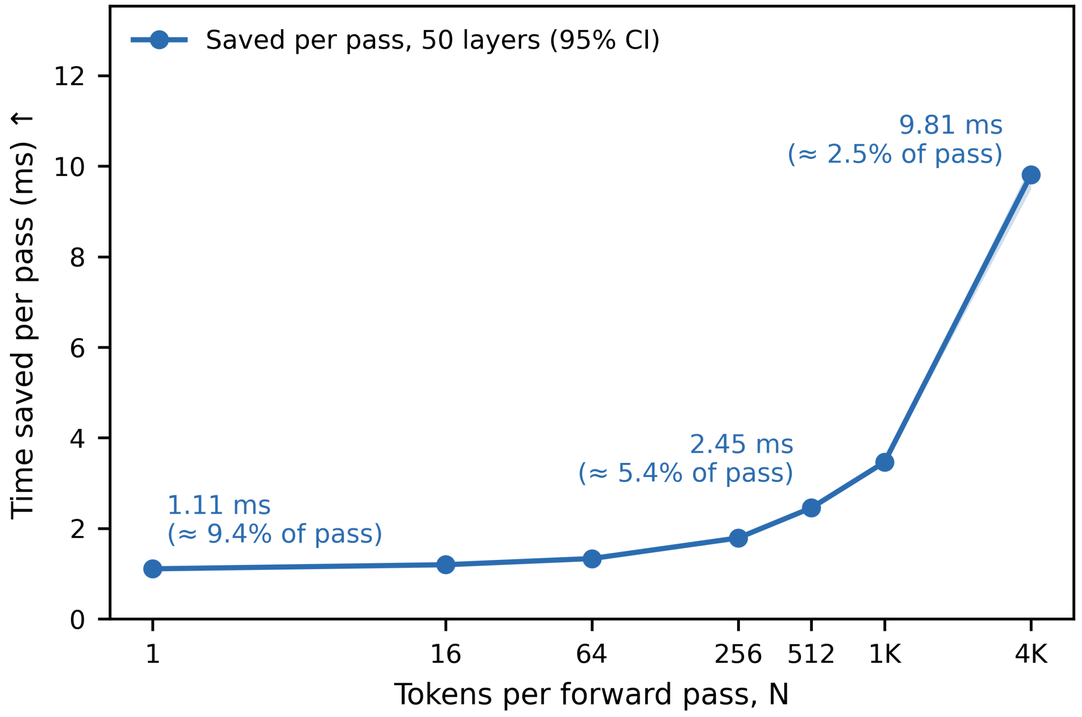
02 Performance



Hardware
NVIDIA GeForce RTX 4090 (24 GB) • 100 interleaved runs

Captured using CUDA Events and benchmarked on a 2-layer slice at full production scale

50 Decoder Layers
Savings scale with tokens per pass, not with context length.



03 Memory Round-Trips

SANDWICH NORM (R1)

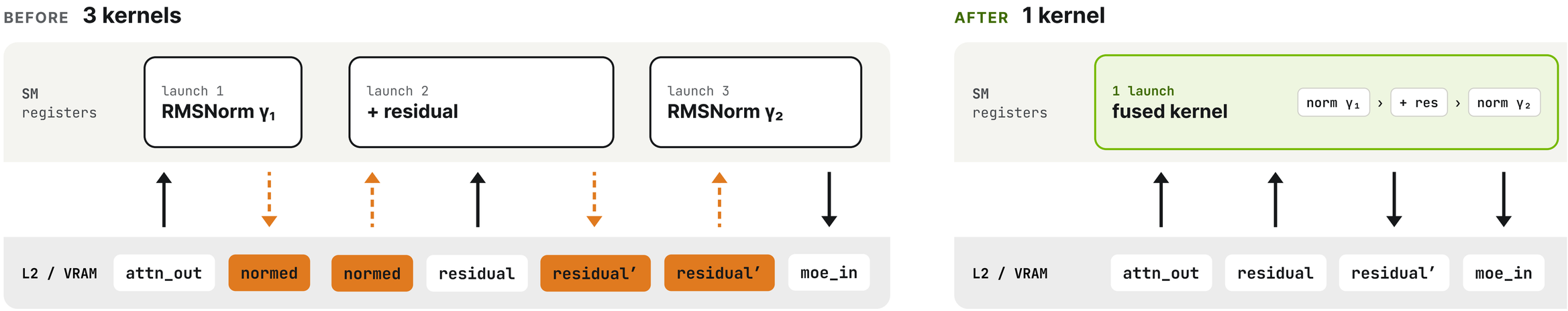


Table 1: Serving Regime (1 Full Wave, L2-Resident)

Metric	TRT-LLM (unfused)	Fused (ours)
Memory Footprint	35.0 KB / token	**20.0 KB / token** (−42.9%)
Intermediate Traffic ($N = 1152$)	17.7 MB (in L2)	**0 MB** (in-register)
Intermediate Storage Tier	L2 Cache	**SM Registers**
Latency at $N = 1152$	28.05 μs	**9.66 μs** (**2.90**×)

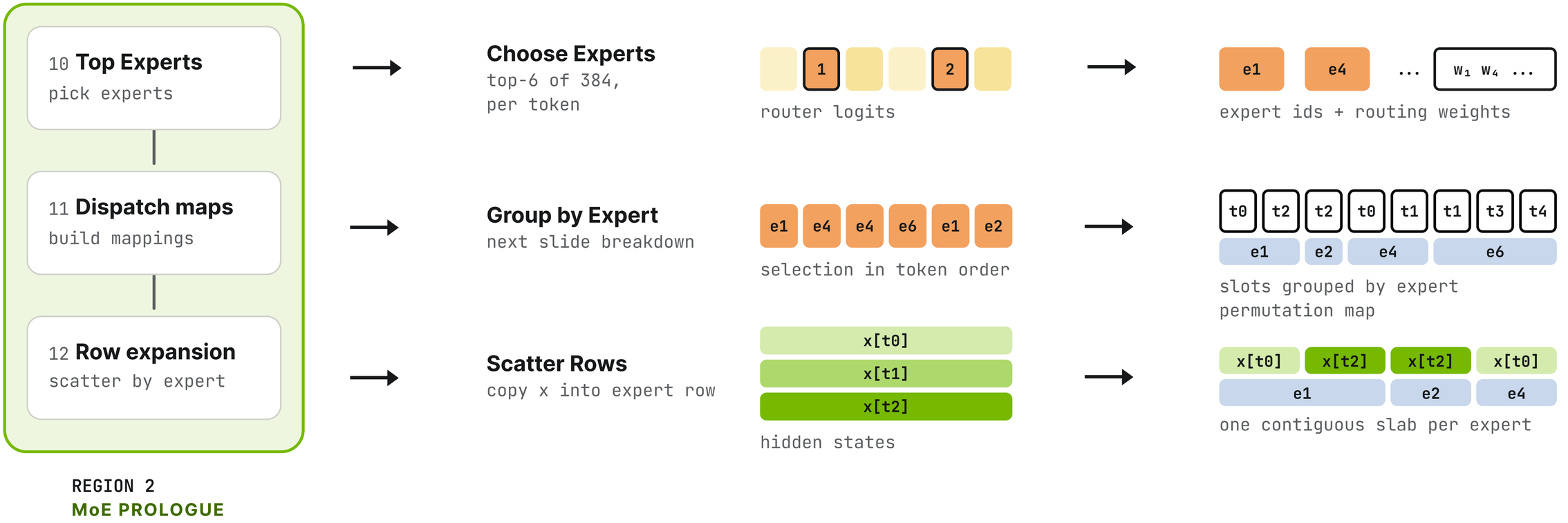
Table 2: Saturated Streaming Regime (16 Waves, N=16K, DRAM-Saturated)

Metric	TRT-LLM (unfused)	Fused (ours)
Working Set ($N = 16$K)	$>$ 335 MB (exceeds L2)	$>$ 335 MB (exceeds L2)
Threadblock Geometry	1024 threads (32 warps)	**160 threads (5 warps)**
Dynamic Shared Memory	5.12 KB / block	**0 B** (in-register)
Peak DRAM Bandwidth	72.1%†	**92.3%** (907 GB/s)
Total Duration (16 Waves)	$\approx$ 830 μs	**326.2 μs** (**2.55**×)

† Weighted average across the 3 unfused baseline kernels.

04 Kernel Breakdown

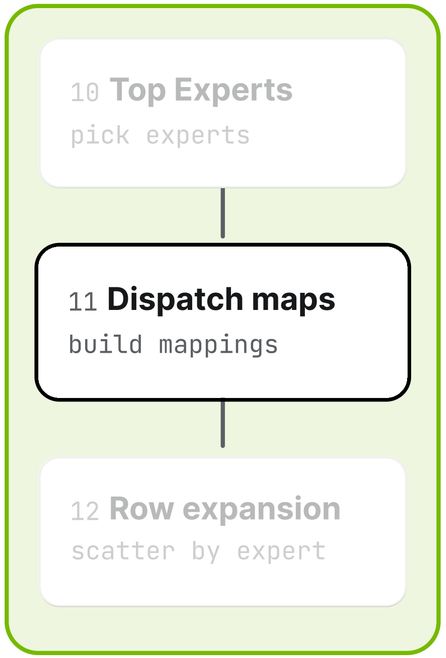
MOE PROLOGUE (R2)



REGION 2
MoE PROLOGUE

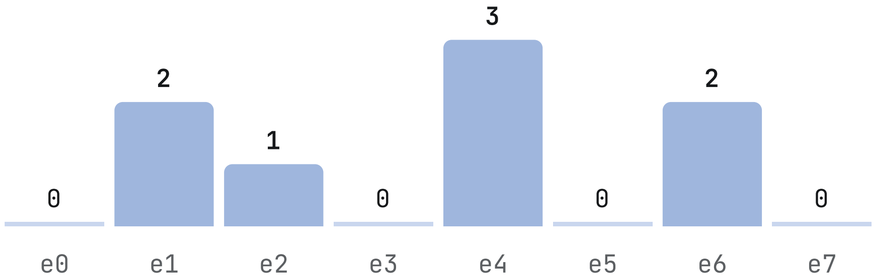
05 Dispatch Maps Operations

MOE PROLOGUE (R2)



REGION 2
MoE PROLOGUE

① COUNT · HISTOGRAM PER EXPERT



`count[e] += 1 per selection`

② SCAN · EXCLUSIVE PREFIX SUM

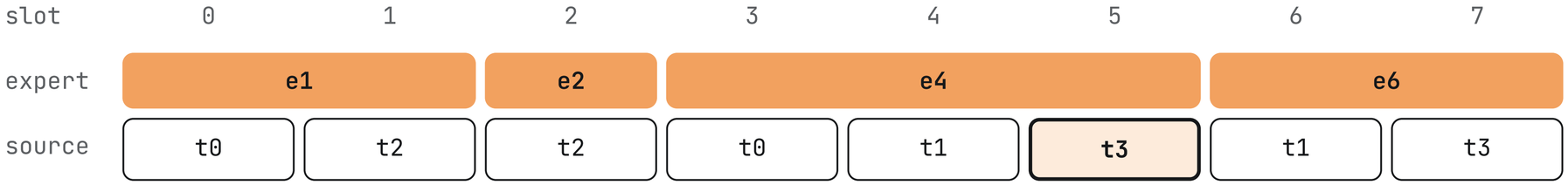
offset[e] = where expert e's block starts. Empty experts get a zero-length block.



`expert_first_token_offset (E + 1)`

③ SCATTER · EACH SELECTION CLAIMS A SLOT

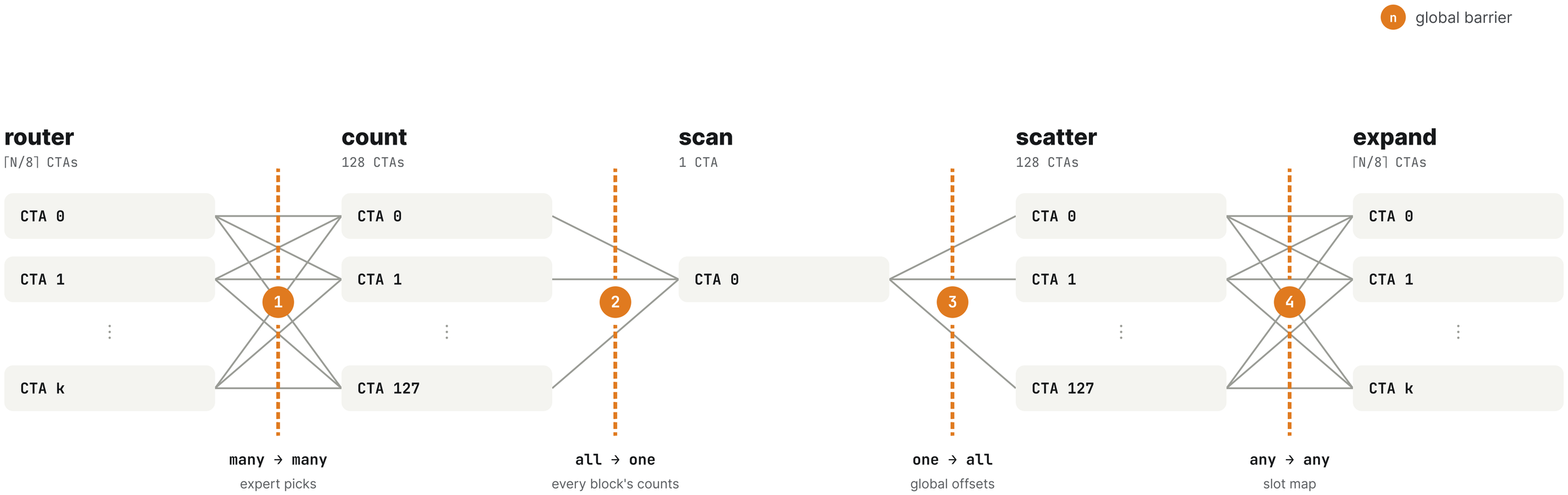
`slot = offset[e] + rank within e` — sel 7 (t3 → e4): 3 + 2 = **5**



`OUT · permuted_row_to_unpermuted_row[slot] = which selection (which token) fills it`

06 All-to-All Dependency

MOE PROLOGUE (R2)



07 Fusion or Fission

MOE PROLOGUE (R2)

128 SMs · n __syncthreads · n global barrier

Fused

1 kernel

router 1 count 2 scan 3 scatter 4 expand

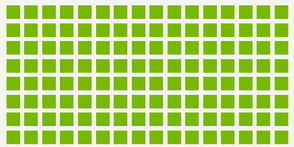
Collapsing 4 global barriers forces 1 CTA : __syncthreads()

1 / 128

Decode

N < 1024
3 kernels

router



128 SMs · Dynamic

1

dispatch maps

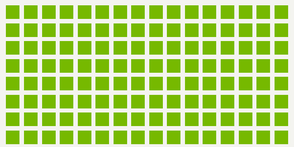
1 SM · small N

count 2 scan 3 scatter

One CTA, so ② ③ become __syncthreads()

4

expand

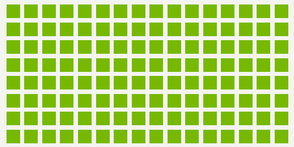


128 SMs · Dynamic

Prefill

N ≥ 1024
5 kernels

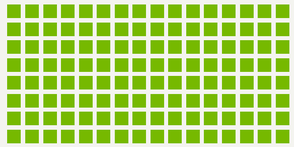
router



128 SMs · Dynamic

1

count



128 SMs

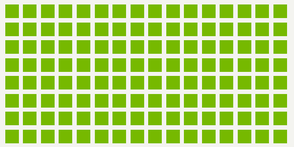
2

scan

1 SM · 128×384, tiny

3

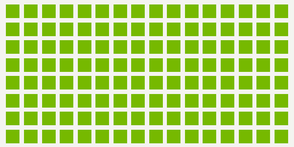
scatter



128 SMs

4

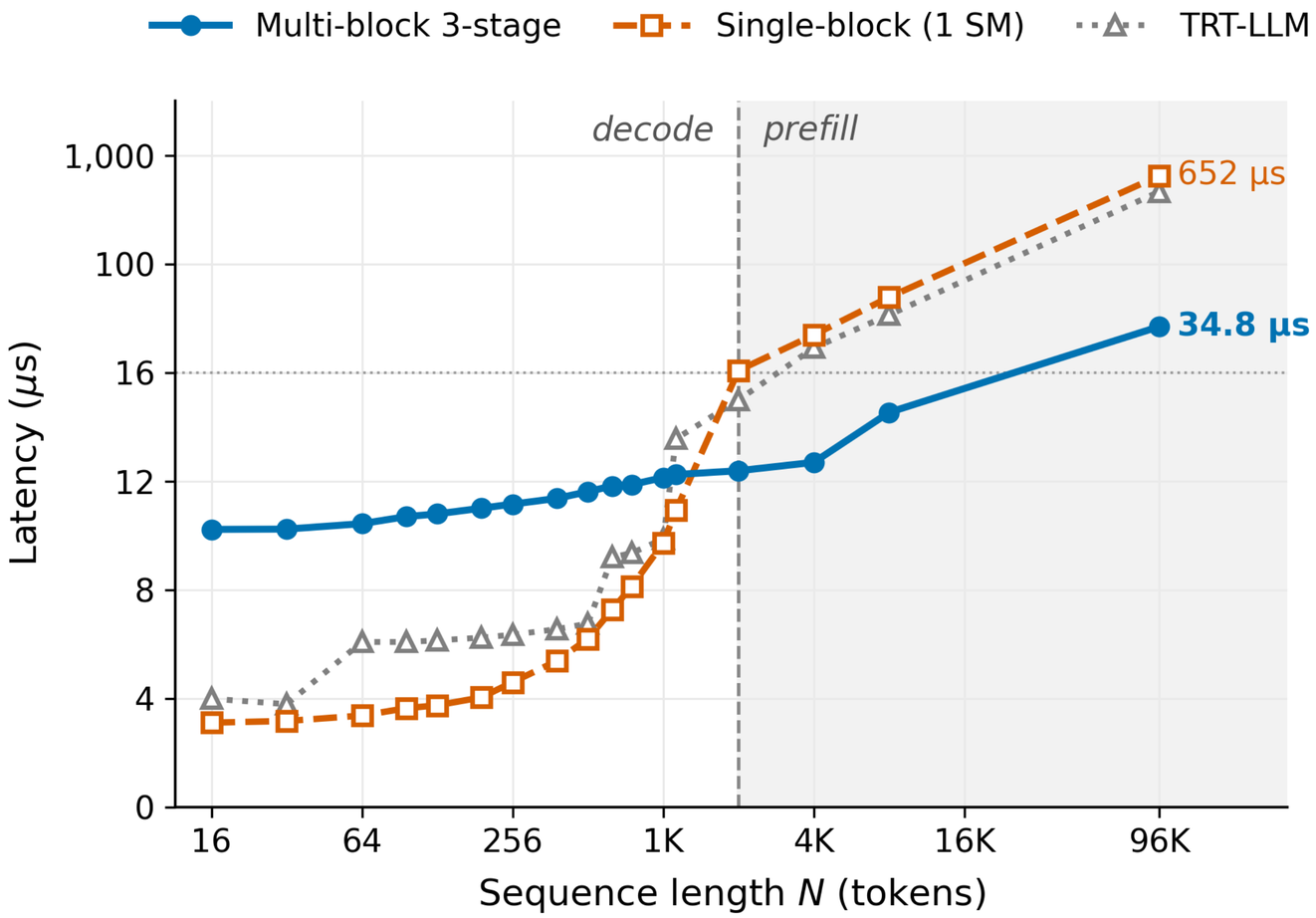
expand



128 SMs · Dynamic

08 Prologue Scaling

MOE PROLOGUE (R2)



Crossover
Below N = 1024, the **single-block** kernel wins (<9µs) by avoiding multi-stage kernel launch overhead.

Above N = 1024, single-block serializes on 1 SM and suffers huge latency penalty at larger batches.

09 Profiling

MOE PROLOGUE (R2)



Table 3: High Occupancy - Memory Bound

Metric	Router
Physical DRAM Throughput	**91.17%** of peak
Compute (SM) Throughput	75.60% of peak
Theoretical SM Occupancy	100.0% (48 warps/SM)
Achieved SM Occupancy	**88.23%**
Active Warps per SM	**42.35** / 48 warps
Executed IPC (Active)	**1.87** inst/cycle

09 Profiling

MOE PROLOGUE (R2)

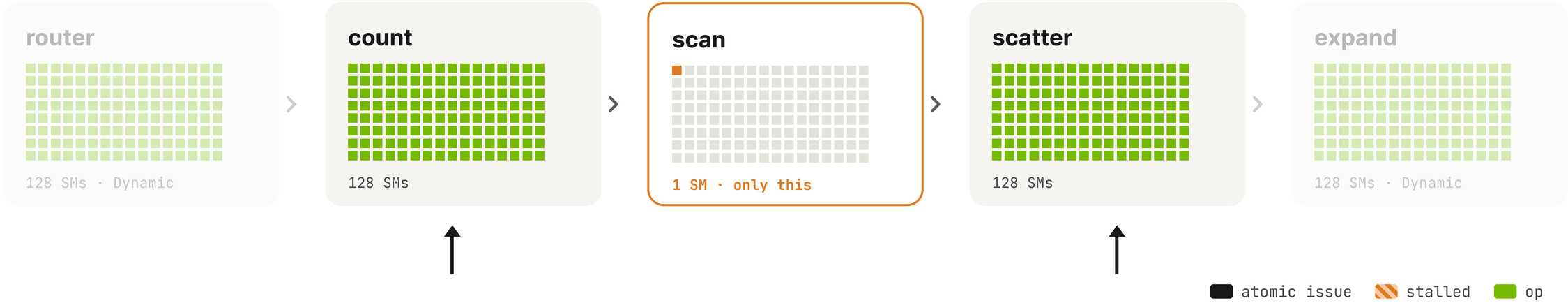


Table 4: Eliminating Redundant Reads

Metric	TRT-LLM	Our Fused
DRAM Bandwidth Saturation	91.6% (898 GB/s)	80.1% (808 GB/s)
Total DRAM Traffic	598.1 MB	**415.4 MB** (−31%)
↪ DRAM Reads (Input)	246.8 MB	**63.2 MB** (−74%)
↪ DRAM Writes (Output)	351.3 MB	351.3 MB (equal)
L2 Cache Hit Rate	68.0%	**85.7%**
Executed Instructions	42.8 M	**10.9 M** (−75%)
Kernel Duration	650.4 μs	**502.3 μs** (1.29×)

Scaling Atomic Operations

MOE PROLOGUE (R2)



Count kernel

fire & forget

```
atomicAdd(&cnt[e], 1);
```

→ red.shared.add

lane 0	red	next op →
lane 1	red	next op →
lane 2	red	next op →
lane 3	red	next op →

No waiting.

Scatter kernel

fetch & wait

```
off = atomicAdd(&ctr[e], 1);
```

→ atom.shared.add

lane 0	atom	▨	write
lane 1	atom	▨	write
lane 2	atom	▨	write
lane 3	atom	▨	write

k lanes on one expert → k serialized waits. More threads, more stalls.

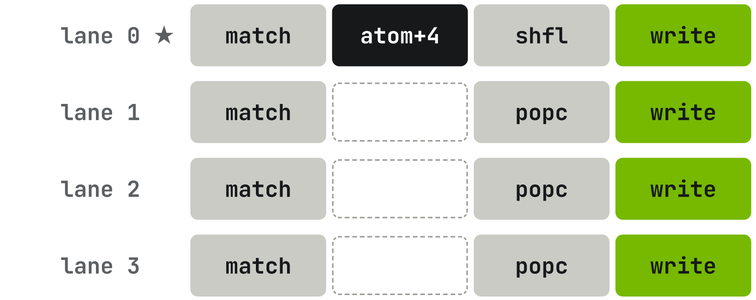
10 Scaling Atomic Operations

MOE PROLOGUE (R2)



Warp Aggregation

```
1  mask = __match_any_sync(ALL, e)  // same expert
2  if leader: base = atomicAdd(&ctr[e], __popc(mask))
3  base = __shfl_sync(mask, base, leader)
4  off  = base + __popc(mask & lanes_below)
```

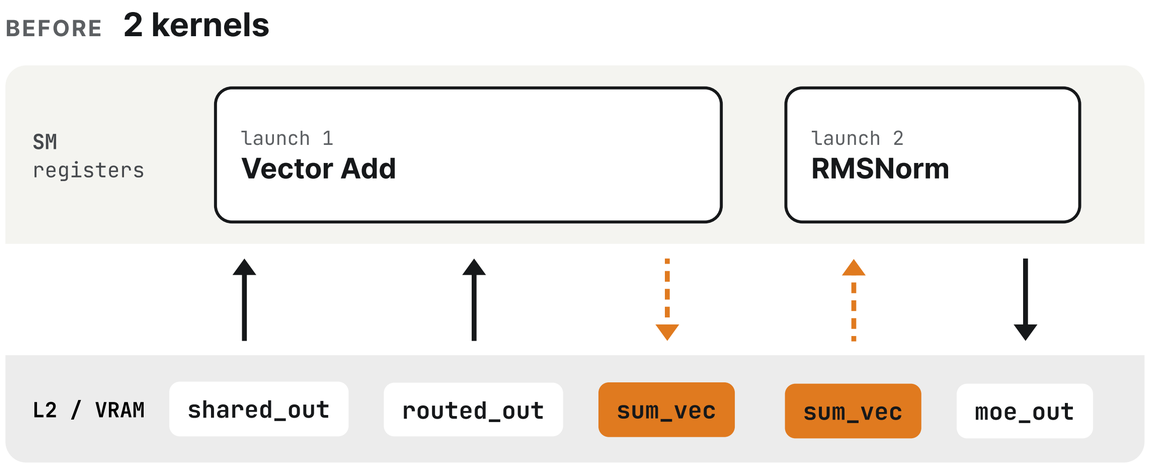


4 atomics → 1

scatter scales to 1024 threads

11 Bypassing the L2 Crossbar

POST-MOE NORM (R3)



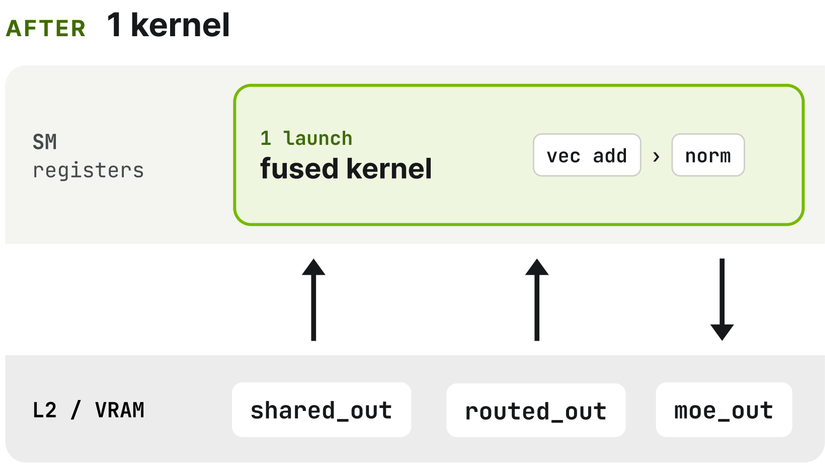


Table 5: Serving Regime (1 Full Wave, L2-Resident)

Metric	TRT-LLM (Baseline)	Fused (Ours)
Intermediate Storage Tier	L2 Cache	**SM Registers**
`sum_vec` Crossbar Traffic	11.8 MB	**0 MB**
L2 Read Hit Rate (Kernel 2)	99.8%	Bypassed
Achieved Occupancy	73.6% (35.3 w/SM)	**91.5%** (43.9 w/SM)
Latency ($N = 1152$)	17.44 μs	**6.28 μs** (2.78×)

Table 6: Saturated Streaming Regime (16 Waves, N=18K, DRAM-Saturated)

Metric	TRT-LLM (Baseline)	Fused (Ours)
Working Set Size	$>$ 370 MB	$>$ 370 MB
Thread Block Geometry	1024 threads (32 warps)	**256 threads** (8 warps)
Scheduler Idle ("No Eligible")	64.0% (Kernel 2)	**8.6%**
Peak DRAM Bandwidth	75.8% (745.0 GB/s)	**95.1%** (934.7 GB/s)
Total Duration (16 Waves)	336.9 μs	**170.0 μs** (1.98×)

12 Integration into TRT-LLM

CALL PATH



L4 Python model

`_torch/models/modeling_kolibri.py`

`torch.ops.trtllm.<op>` in `forward()`

op missing → falls back to plain PyTorch

L3 Fake tensor op `trace time`

`_torch/custom_ops/cpp_custom_ops.py`

`@register_fake("trtllm::<op>")`

shapes only: torch.compile / CUDA Graphs trace without a CPU sync

L2 Torch custom op

`cpp/tensorrt_llm/thop/moe/*/<op>.cpp`

`TORCH_LIBRARY_FRAGMENT(trtllm, m)`

checks layout + contiguity · runs on the current CUDA stream · auto-globbed into libth_common.so

L1 CUDA kernels

`cpp/tensorrt_llm/kernels/moe/*/<kernel>.cu`

The kernels from this talk: fused norms, dispatch, expansion

C++ launcher in `tensorrt_llm::kernels`

TECHNICAL PRESENTATION

END

By Amine Kina
M.Sc. at University of Bremen